REPORTE DE PRÁCTICA NO. 3

Construcción de un Autómata Finito Determinista

ALUMNO:Arrazola Ibarra Juan Pablo
Dr. Eduardo Cornejo-Velázquez

1. Introducción

Los Autómatas Finitos son máquinas teóricas utilizadas en la teoría de la computación para modelar sistemas que pueden encontrarse en un número finito de estados. Estos modelos permiten representar procesos que cambian de estado dependiendo de los símbolos que reciben como entrada. La salida de un autómata finito generalmente indica si una cadena de entrada es aceptada o rechazada por el sistema.
Los Autómatas Finitos Deterministas (AFD) representan un tipo particular de autómata en el cual para cada estado y cada símbolo del alfabeto existe exactamente una transición posible. Esto garantiza que el comportamiento del sistema sea completamente determinista, es decir, que siempre exista un único camino de ejecución para cada cadena de entrada.
Los AFD tienen aplicaciones importantes dentro de la informática, particularmente en el diseño de compiladores, el análisis léxico, el reconocimiento de patrones y el procesamiento de lenguajes formales. Estos modelos permiten determinar si una cadena pertenece o no a un lenguaje específico definido sobre un alfabeto determinado.
En esta práctica se desarrolla un programa utilizando un lenguaje de programación de alto nivel que implementa un Autómata Finito Determinista. El programa recibe como entrada la definición del autómata y un conjunto de cadenas de símbolos, evaluando si cada una de ellas es aceptada o rechazada por el autómata.

2. Marco teórico

Los lenguajes formales y la teoría de autómatas forman parte fundamental de la teoría de la computación. Estas áreas estudian los modelos matemáticos utilizados para describir y analizar lenguajes y sistemas computacionales [1].
Un Autómata Finito Determinista se define formalmente como una quíntupla:

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

donde:

- Σ es el alfabeto de entrada.
- Q es el conjunto finito y no vacío de estados.
- δ es la función de transición definida como $\delta : Q \times \Sigma \to Q$.
- q_0 es el estado inicial.
- F es el conjunto de estados finales o de aceptación.

Sea $A = (\Sigma, Q, \delta, q_0, F)$ un AFD y sea $w = w_1 w_2 ... w_n$ una cadena de símbolos donde cada símbolo pertenece al alfabeto Σ.
El autómata acepta la cadena si existe una secuencia de estados:

$$r_0, r_1, ..., r_n$$

que cumple las siguientes condiciones:

1. $r_0 = q_0$
2. $\delta(r_i, w_{i+1}) = r_{i+1}$ para $i = 0, 1, ..., n-1$
3. $r_n \in F$

La primera condición establece que el procesamiento comienza en el estado inicial. La segunda condición indica que el autómata cambia de estado de acuerdo con la función de transición dependiendo del símbolo leído. Finalmente, la tercera condición establece que la cadena es aceptada si al finalizar la lectura se alcanza un estado de aceptación.
Los autómatas finitos son utilizados ampliamente en el desarrollo de compiladores, particularmente en la fase de análisis léxico, donde se emplean para reconocer patrones y validar cadenas dentro de un lenguaje de programación [2].

3. Herramientas empleadas

Para la realización de esta práctica se utilizaron las siguientes herramientas:

1. Computadora personal para el desarrollo y ejecución del programa.
2. Lenguaje de programación de alto nivel para implementar el Autómata Finito Determinista.
3. Editor de código o entorno de desarrollo para escribir y probar el programa.
4. Compilador o intérprete del lenguaje utilizado para ejecutar el programa.

4. Desarrollo

De acuerdo con el procedimiento de la práctica, se desarrolló un programa en Python que implementa un Autómata Finito Determinista (AFD). El programa recibe como entrada la descripción del autómata (estados, alfabeto, transiciones, estado inicial y estados de aceptación) y posteriormente evalúa un conjunto de cadenas para determinar si son aceptadas o rechazadas por el autómata.

Procedimiento

El programa funciona de la siguiente manera:

1. Se leen los valores iniciales del autómata: número de estados (N), tamaño del alfabeto (S), número de transiciones (D), estado inicial (q_0), número de estados finales (T) y número de cadenas a evaluar (C).
2. Se lee el alfabeto Σ.
3. Se leen los estados de aceptación F.
4. Se almacenan las transiciones del autómata en una estructura de datos.
5. Para cada cadena de entrada, el programa recorre símbolo por símbolo aplicando la función de transición.
6. Al terminar la cadena, se verifica si el estado final pertenece al conjunto de estados de aceptación.

Ejemplo del Autómata

En la Figura **??** se muestra el autómata utilizado como ejemplo en la práctica, el cual reconoce palabras que tienen una cantidad par de 0's después del último 1.

- $Q = \{1, 2, 3\}$
- $\Sigma = \{0, 1\}$
- Estado inicial: $q_0 = 1$
- Estado de aceptación: $F = \{2\}$

Implementación en Python

A continuación se muestra el código utilizado para implementar el Autómata Finito Determinista.

Listing 1: Implementación del AFD en Python.

```
# Leer datos iniciales
N, S, D, q0, T, C = map(int, input().split())

alfabeto = input().split()

finales = list(map(int, input().split()))

# Guardar transiciones
transiciones = {}

for _ in range(D):
    i, x, j = input().split()
    transiciones[(int(i), x)] = int(j)
```

```python
# Revisar cadenas
for _ in range(C):
    cadena = input().strip()
    estado = q0

    for simbolo in cadena:
        estado = transiciones[(estado, simbolo)]

    if estado in finales:
        print(cadena, "ACEPTADA")
    else:
        print(cadena, "RECHAZADA")
```

Ejemplo de entrada

```
3 2 6 1 1 4
0 1
2
1 0 1
1 1 2
2 0 3
2 1 2
3 0 2
3 1 2
101
0
11
1000100001
```

Resultado obtenido

```
101 ACEPTADA
0 RECHAZADA
11 ACEPTADA
1000100001 ACEPTADA
```

5. Cuestionario

¿Qué paradigma de programación utilizaste para la implementación del AFD?, ¿Cuáles elementos del lenguaje de programación utilizaste?
Para la implementación del Autómata Finito Determinista se utilizó el paradigma de programación estructurada. Este paradigma permite organizar el programa mediante estructuras de control como ciclos, condicionales y funciones, lo que facilita la comprensión del algoritmo y la correcta implementación del autómata.
¿Qué estructuras de datos utilizaste para implementar los componentes del AFD?
Para representar los componentes del autómata se utilizaron estructuras de datos como arreglos y matrices. Los estados, el alfabeto y los estados de aceptación pueden almacenarse en arreglos, mientras que las transiciones del autómata pueden representarse mediante una matriz o tabla de transiciones que indique el siguiente estado dependiendo del estado actual y del símbolo leído.
¿Qué elementos del lenguaje de programación te permitieron implementar las funciones de transición?, ¿cuáles son las limitaciones del programa que construiste?
Las funciones de transición se implementaron utilizando estructuras condicionales y matrices que almacenan las transiciones entre estados. Esto permite que el programa determine el siguiente estado del autómata dependiendo del estado actual y del símbolo de entrada.
Entre las principales limitaciones del programa se encuentra la dependencia del formato de entrada definido para describir el autómata. Además, el tamaño del autómata que puede manejar el programa depende de los límites establecidos en las estructuras de datos utilizadas.
¿Qué condiciones se deben cumplir para que el programa pueda evaluar cualquier AFD?
Para que el programa pueda evaluar correctamente cualquier Autómata Finito Determinista es necesario que todos sus componentes estén correctamente definidos. Esto incluye el conjunto de estados, el alfabeto de entrada, la función de transición, el estado inicial y el conjunto de estados de aceptación. Además, la función de transición debe estar completamente definida para cada combinación de estado y símbolo del alfabeto.

6. Conclusiones

En esta práctica se logró comprender el funcionamiento de los Autómatas Finitos Deterministas y su aplicación para el reconocimiento de cadenas dentro de un lenguaje formal. Asimismo, se desarrolló un programa capaz de analizar diferentes cadenas de entrada y determinar si son aceptadas o rechazadas por el autómata definido.

El desarrollo de esta práctica permitió reforzar conocimientos relacionados con la teoría de la computación y los lenguajes formales, además de fortalecer habilidades de programación necesarias para implementar modelos matemáticos dentro de un programa computacional.

Referencias Bibliográficas

References

[1] Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). *Lenguajes formales y teoría de autómatas*. Editorial Alfaomega. Argentina.

[2] Ruiz Catalán, J. (2010). *Compiladores: teoría e implementación*. Editorial Alfaomega. México.

[3] Brookshear, J. G. (1995). *Teoría de la Computación: Lenguajes formales y autómatas*.